

Ex.No-13

Experiment 13: Write a Python Program to Implement the Minimax Algorithm for Gaming

Aim

The aim of this experiment is to write a Python program to implement the Minimax algorithm for a game and determine the best possible move for the player.

Objectives

- To understand the Minimax algorithm.
- To write a Python program for Minimax.
- To find the best move in a game.
- To display the optimal move.

Algorithm

1. Start.
2. Define the game state and possible moves.
3. Check if the game has reached a terminal state.
4. If it is the maximizing player's turn, choose the maximum score.
5. If it is the minimizing player's turn, choose the minimum score.
6. Repeat recursively until the game ends.
7. Display the best move and its score.
8. Stop.

Program (Python)

```
# Minimax Algorithm
```

```
scores = {  
    'A': 3,  
    'B': 5,  
    'C': 2,  
    'D': 9  
}
```

```
def minimax(states, is_max):
```

```

if len(states) == 1:
    return scores[states[0]]

mid = len(states) // 2

left = minimax(states[:mid], not is_max)

right = minimax(states[mid:], not is_max)

if is_max:
    return max(left, right)

else:
    return min(left, right)

states = ['A', 'B', 'C', 'D']
result = minimax(states, True)
print("Best Score:", result)

```

Sample Input

Game States:

A = 3

B = 5

C = 2

D = 9

Output

```

IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI Ex-13.py =====
Best Score: 3
>>>

```

Result

The Python program for the Minimax Algorithm was successfully implemented. The program evaluated the game states recursively and displayed the best possible score for the maximizing player.

Ex.No-14

Experiment : Write a Python Program to Implement Alpha-Beta Pruning Algorithm for Gaming

Aim

The aim of this experiment is to write a Python program to implement the Alpha-Beta Pruning algorithm for a game and find the best move by reducing unnecessary searches.

Objectives

- To understand the Alpha-Beta Pruning algorithm.
- To write a Python program for Alpha-Beta Pruning.
- To find the best move efficiently.
- To display the optimal score.

Algorithm

1. Start.
2. Define the game tree and leaf node values.
3. Initialize $\alpha = -\infty$ and $\beta = +\infty$.
4. Traverse the game tree using Minimax.
5. Update α and β values.
6. Prune branches when $\alpha \geq \beta$.
7. Display the best score.
8. Stop.

Program (Python)

```
import math

values = [3, 5, 6, 9, 1, 2, 0, -1]

def alphabeta(depth, index, maximizing, alpha, beta):
    if depth == 3:
        return values[index]
    if maximizing:
        best = -math.inf
        for i in range(2):
            val = alphabeta(depth + 1, index * 2 + i, False, alpha, beta)
```

```

        best = max(best, val)

        alpha = max(alpha, best)

        if beta <= alpha:

            break

    return best

else:

    best = math.inf

    for i in range(2):

        val = alphabeta(depth + 1, index * 2 + i, True, alpha, beta)

        best = min(best, val)

        beta = min(beta, best)

        if beta <= alpha:

            break

    return best

result = alphabeta(0, 0, True, -math.inf, math.inf)

print("Optimal Value:", result)

```

Sample Input

Leaf Node Values:

3 5 6 9 1 2 0 -1

Output

Result

The Python program for the Alpha-Beta Pruning Algorithm was successfully implemented. The program efficiently evaluated the game tree by pruning unnecessary branches and displayed the optimal value correctly.

Ex.No-15

Experiment : Write a Python Program to Implement a Decision Tree

Aim

The aim of this experiment is to write a Python program to implement a Decision Tree for classifying data based on the given features.

Objectives

- To understand the Decision Tree algorithm.
- To write a Python program for Decision Tree classification.
- To train the model using sample data.
- To predict the class of new data.

Algorithm

1. Start.
2. Import the required libraries.
3. Prepare the training data and target values.
4. Create a Decision Tree classifier.
5. Train the model using the training data.
6. Predict the class for the test data.
7. Display the prediction.
8. Stop.

Program (Python)

```
# Decision Tree Implementation without sklearn
print("===== Decision Tree Classification =====")
print("Predict whether a person will buy a product")
age = int(input("Enter Age: "))
salary = int(input("Enter Annual Salary: "))
student = input("Are you a Student? (Yes/No): ")
credit = input("Credit Rating (Fair/Excellent): ")
if age < 30:
    if student.lower() == "yes":
```

```

        decision = "Yes"
    else:
        decision = "No"
elif age >= 30 and age <= 40:
    if salary >= 50000:
        if credit.lower() == "fair":
            decision = "Yes"
        else:
            decision = "Yes"
    else:
        decision = "No"
else:
    if credit.lower() == "excellent":
        decision = "No"
    else:
        decision = "Yes"
print("\n----- Decision Tree Result -----")
print("Age      :", age)
print("Salary    :", salary)
print("Student    :", student)
print("Credit Rating :", credit)
print("Prediction  :", decision)

```

Sample Input

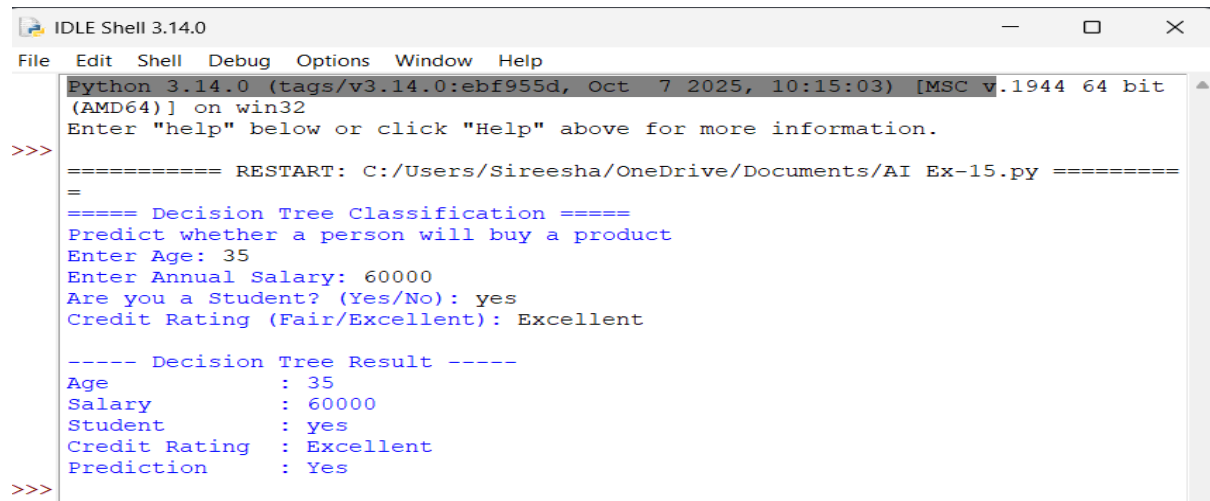
Training Data:

Age	Salary	Buy
25	50000	No
35	65000	Yes
45	80000	Yes
20	30000	No

Test Data:

30 60000

Output



```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> ===== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI Ex-15.py =====
=
===== Decision Tree Classification =====
Predict whether a person will buy a product
Enter Age: 35
Enter Annual Salary: 60000
Are you a Student? (Yes/No): yes
Credit Rating (Fair/Excellent): Excellent

----- Decision Tree Result -----
Age           : 35
Salary        : 60000
Student       : yes
Credit Rating : Excellent
Prediction    : Yes
>>>
```

Result

The Python program for the Decision Tree algorithm was successfully implemented. The model was trained using the given data and correctly predicted the class for the test data.

Ex.No-16

Experiment : Write a Python Program to Implement a Feed Forward Neural Network

Aim

The aim of this experiment is to write a Python program to implement a Feed Forward Neural Network for processing input data and producing the corresponding output.

Objectives

- To understand the Feed Forward Neural Network.
- To write a Python program for a Feed Forward Neural Network.
- To process input through hidden and output layers.
- To display the output of the network.

Algorithm

1. Start.
2. Initialize the input values, weights, and biases.
3. Calculate the output of the hidden layer.
4. Apply the sigmoid activation function.

5. Calculate the output of the output layer.
6. Apply the sigmoid activation function.
7. Display the final output.
8. Stop.

Program (Python)

```
import math

def sigmoid(x):
    return 1 / (1 + math.exp(-x))

x1 = 0.5
x2 = 0.8
w1 = 0.2
w2 = 0.4
w3 = 0.3
w4 = 0.7
w5 = 0.6
w6 = 0.9
b1 = 0.5
b2 = 0.5

h1 = sigmoid(x1 * w1 + x2 * w2 + b1)
h2 = sigmoid(x1 * w3 + x2 * w4 + b1)
output = sigmoid(h1 * w5 + h2 * w6 + b2)

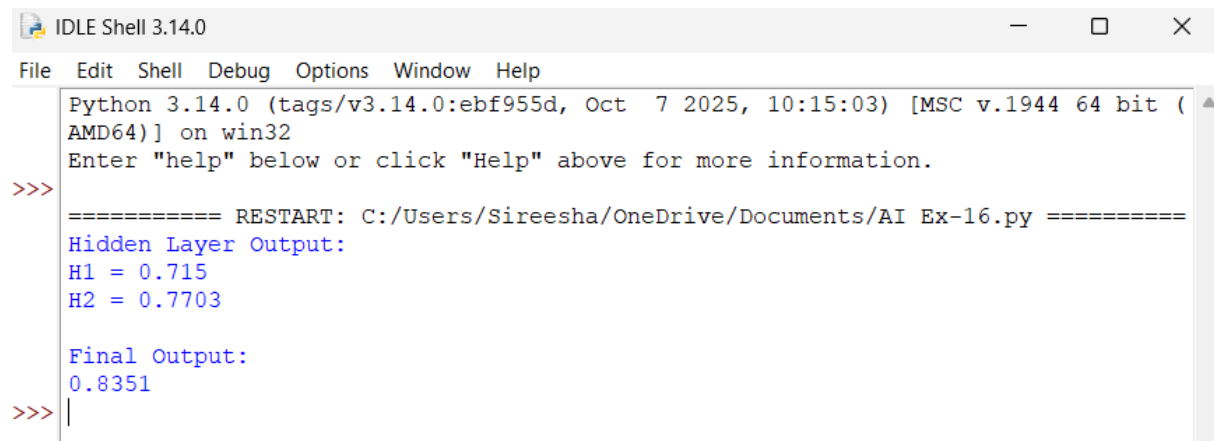
print("Hidden Layer Output:")
print("H1 =", round(h1, 4))
print("H2 =", round(h2, 4))
print("\nFinal Output:")
print(round(output, 4))
```

Sample Input

Input 1 = 0.5

Input 2 = 0.8

Output



```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI Ex-16.py =====
Hidden Layer Output:
H1 = 0.715
H2 = 0.7703

Final Output:
0.8351
>>> |
```

Result

The Python program for the Feed Forward Neural Network was successfully implemented. The network processed the input values through the hidden layer and output layer using the sigmoid activation function, and the final output was displayed successfully.